

Diseño de una Unidad Aritmética y Lógica

Montoya Sánchez Máximo Alexander

1906

Resumen

La implementación de una Unidad Aritmética y Lógica optimiza los procesos de cálculo en las operaciones comunes como las sumas, restas, multiplicaciones y también en las operaciones lógicas AND, OR, NOT, dando el resultado sin afectar otros procesos que se realicen en paralelo.

Introducción

Las operaciones aritméticas y lógicas en un procesador gastan recursos importantes al momento de sacar resultados debido a la optimización de estos procesos se implementó una Unidad Aritmética y Lógica que es circuito digital encargado de quitarle todo el gasto de recursos a un procesador, optimizando los tiempos de carga y las operaciones que se pueden realizar.

Desarrollo

El diseño de la Unidad Aritmética y Lógica se realizó con el ambiente de desarrollo “Vivado” y con “systemverilog” como lenguaje de descripción, se utilizó un código “modular” ya que cada una de las operaciones que la conforman se realizaron en un archivo individual, se avanzó así para tener mejor

administradas las operaciones, aunque se tenga un código más largo.

El archivo principal “Alu_sel” (top) es la selección de operación con una secuencia de tres bits como se muestra en la [Tabla 1](#).

Operación	Salida
000	$A + B$
001	$A - B$
010	$A \& B$
011	$A B$
100	$A \wedge B$
101	$\sim A$
110	$A \ll 1$
111	$A \gg 1$

Tabla 1. Operaciones de la ALU.

Se utilizó una instanciación de todas las operaciones al archivo principal y un “case” para poder seleccionar la salida, las variables utilizadas fueron un total de 8; 3 entradas de 4 bits cada una que serán la entrada “a” y “b” y el selector de operación “sel” y 5 salidas una de 4 bits que es el “out_alu” para la salida de las operaciones y las demás de 1 bit las cuales son las “flags” que indican si ocurrió un acarreo (carry en la suma o borrow en la resta), desborde (overflow

por cambio de signo o el resultado se desborda), si el resultado es cero (zero), o si es negativo (negative), a continuación, se describirá el funcionamiento de las operaciones. [12]

Suma y Resta

La suma de 4 bits tiene un proceso simple creando una variable interna de 5 bits “sum_in” para poder hacer la operación y guardar el acarreo “c” según lo requiera, después vienen las partes de asignación de valores de las banderas, en esta parte se utilizó una asignación “condicional” ya que para que se active una bandera se necesitan diferentes condiciones [4]. Con la resta se usó el mismo código solo se cambió la variable interna y las condiciones de las banderas [5].

Operadores lógicos

El código del operador lógico AND y de los siguientes es más simple ya que no se utilizan todas las banderas aparte no se necesita otra variable interna, las

Desplazamientos Lógicos

Como se mencionó los códigos siguientes serán más sencillos, aquí solo se usa el operador “<<” para “multiplicar” por dos y “>>” para “dividir” entre dos, las banderas también tienen sus propias condiciones de activación.

Resultados de la simulación

Para poder probar el código realizado se necesita crear un nuevo archivo que funcionara como banco de pruebas (Test Bench), en este archivo se necesita instanciar el archivo principal “Alu_sel” para poder asignar valores a las entradas y que detecte todas las salidas definidas en la [Figura 2](#) se muestra el código implementado para la simulación.

En la [Figura 1](#) está la simulación de las entradas propuestas, en la gráfica de tiempo se puede ver el selector de 3 bits, la salida de las operaciones y las banderas (flags). Las operaciones se realizan correctamente, aunque en la



Figura 1 Gráfica de la simulación

banderas que no se utilizan se les asigna un cero y las otras tienen sus respectivas condiciones de activación [8].

salida de la resta hay una salida en alta impedancia (z).

Conclusiones

De acuerdo con los resultados de la simulación hay algunos puntos de mejora en los códigos de las operaciones, específicamente en las condiciones de las banderas, no todas las banderas se activan correctamente y también el código de la resta hay un error en la entrada “a” no detecta bien el número binario colocado y a la salida da alta impedancia pero el objetivo de diseñar, implementar y simular una ALU se logró teniendo las 8 operaciones integradas en un selector.

Apéndice

```

1 module ALU_tb;
2     logic [3:0] a,b;
3     logic [2:0] sel;
4     logic [3:0] out_alu;
5     logic n,z,o,c;
6
7     ALU_selec tb(.a);
8
9     initial
10    begin
11        sel=3'b000;
12        a=4'b0010;
13        b=4'b0011;
14        #30;
15        sel=3'b001;
16        a=4'b1010;
17        b=4'b0110;
18        #30;
19        sel=3'b010;
20        a=4'b0100;
21        b=4'b0001;
22        #30;
23        sel=3'b011;
24        a=4'b0100;
25        b=4'b0001;
26        #30;
27        sel=3'b100;
28        a=4'b0100;
29        b=4'b0001;
30        #30;
31        sel=3'b101;
32        a=4'b0100;
33        #30;
34        sel=3'b110;
35        a=4'b0100;
36        #30;
37        sel=3'b111;
38        a=4'b0100;
39        #30;
40    end

```

Figura 2 Código del test bench.

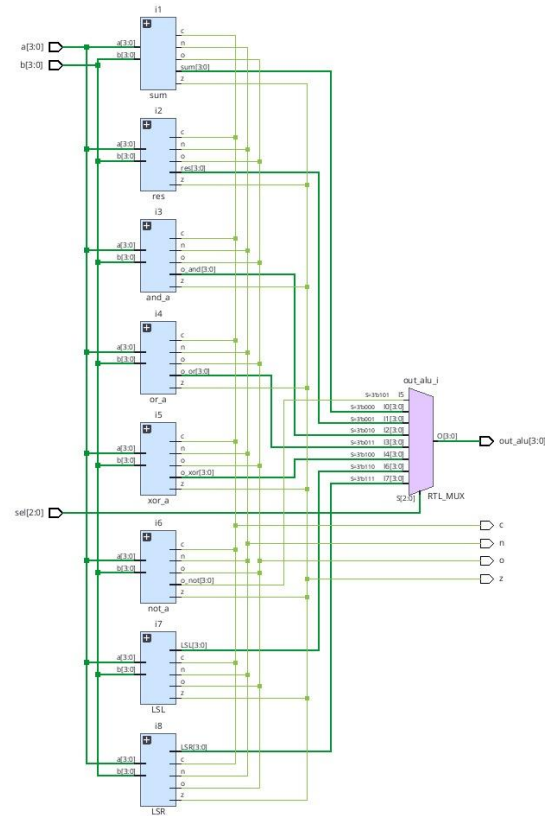


Figura 3 Diagrama RTL.

```

1 module sum(
2     input logic [3:0] a,b,
3     output logic [3:0] sum,
4     output logic n,z,o,c
5 );
6     logic [4:0] sum_int;
7     assign sum_int = a+b;
8     assign sum = sum_int[3:0];
9     assign c = sum_int [4];
10    assign o=(sum_int[3:0]==4'b0000);
11    assign n = 1'b0;
12    assign z=(sum_int[4:0]==5'b00000);
13 endmodule

```

Figura 4

```

module res(
    input [3:0] a,b,
    output [3:0] res,
    output logic n,z,o,c
);
    logic [4:0] res_int;
    assign res_int = a-b;
    assign sum = res_int[3:0];
    assign c= res_int [4];
    assign o=(~a[3] & b[3] & res_int[3]) | (a[3] & ~b[3] & ~res_int[3]);
    assign z=(res==4'b0000);
    assign n=res_int[3];
endmodule

```

Figura 5

```

module and_a(
    input logic [3:0] a,b,
    output logic [3:0] o_and,
    output logic n,z,o,c
);
    assign o_and = a & b;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (o_and == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 6

```

module xor_a(
    input logic [3:0] a,b,
    output logic [3:0] o_xor,
    output logic n,z,o,c
);
    assign o_xor = a ^ b;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (o_xor == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 7

```

module not_a(
    input logic [3:0] a,b,
    output logic [3:0] o_not,
    output logic n,z,o,c
);
    assign o_not = ~a;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (o_not == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 8

```

module LSL(
    input logic [3:0] a,b,
    output logic [3:0] LSL,
    output logic n,z,o,c
);
    assign LSL = a << 1;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (LSL == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 9

```

module LSR(
    input logic [3:0] a,b,
    output logic [3:0] LSR,
    output logic n,z,o,c
);
    assign LSR = a >> 1;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (LSR == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 10

```

module or_a(
    input logic [3:0] a,b,
    output logic [3:0] o_or,
    output logic n,z,o,c
);
    assign o_xor = a | b;
    assign c = 1'b0;
    assign o = 1'b0;
    assign z = (o_or == 4'b0000);
    assign n = 1'b0;
endmodule

```

Figura 11

```

module ALU_selec(
    input logic [3:0] a,b,
    input logic [2:0] sel,
    output logic [3:0] out_alu,
    output logic n,z,o,c
);
    logic [3:0] sum,res,o_and,o_or,o_xor,o_not,LSL,LSR;
    logic n_f,z_f,o_f,c_f;

    sum i1(.a(a),.b(b),.sum(sum),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    res i2(.a(a),.b(b),.res(res),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    and_a i3(.a(a),.b(b),.o_and(o_and),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    or_a i4(.a(a),.b(b),.o_or(o_or),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    xor_a i5(.a(a),.b(b),.o_xor(o_xor),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    not_a i6(.a(a),.b(b),.o_not(o_not),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    LSL i7(.a(a),.b(b),.LSL(LSL),.n(n_f),.z(z_f),.o(o_f),.c(c_f));
    LSR i8(.a(a),.b(b),.LSR(LSR),.n(n_f),.z(z_f),.o(o_f),.c(c_f));

    always_comb
    case(sel)
        3'b000:
            begin
                out_alu=sum;
                n=n_f;
                z=z_f;
                o=o_f;
                c=c_f;
            end
        3'b001:
            begin
                out_alu=res;
                n=n_f;
                z=z_f;
                o=o_f;
                c=c_f;
            end
        3'b010:
            begin
                out_alu=o_and;
                n=n_f;
                z=z_f;
                o=o_f;
                c=c_f;
            end
        3'b011:
            begin
                out_alu=o_or;
                n=n_f;
                z=z_f;
                o=o_f;
                c=c_f;
            end
        3'b100:
    end

```

Figura 12